MatatuPulse — Build Plan

Nov 7, 2026 · @Jay

Goal and scope

Ship one working demo: a commuter signs in, opens Route 100, sees live fares and verified alerts, reports a fare hike, and the Signal panel shows 200 raw reports becoming 12 verified events.

Must ship

- Phone + OTP sign-in
- Route list with status pills, route detail with sacco fares and commuter reports
- Report flow with duplicate merging and verify-at-3
- "Still accurate?" and "Cleared?" votes
- Signal panel with the funnel, raw SMS, verified events and alert log

Nice to have: inbound SMS from a real gateway, push or SMS alerts to subscribers.

Cut: maps, GPS, payments, admin moderation UI, native apps.

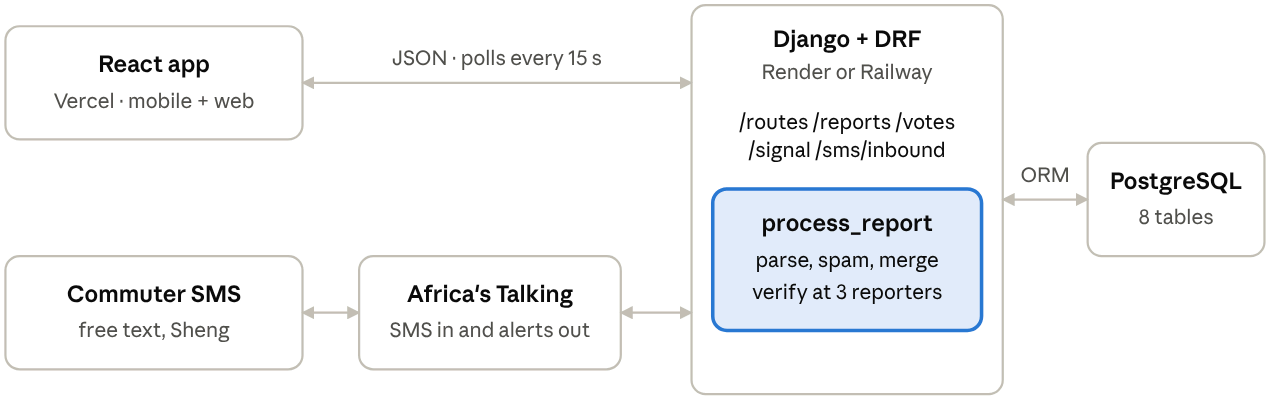
Stack

Django REST API + PostgreSQL behind one React app; no queue, no websockets. Report processing runs inline on each request and the app polls every 15 s.

Layer	Choice	Why
API	Django 5 + Django REST Framework	Admin for free, fast CRUD
Auth	Phone OTP + djangorestframework-simplejwt	No passwords; JWT in the React app
Database	PostgreSQL (SQLite fine locally)	Free tier on Render, Railway or Supabase
Front-end	React + Vite + Tailwind	Matches the designs one-to-one
Data fetching	TanStack Query, 15 s polling	Live feel without websockets

Layer	Choice	Why
SMS in/out	Africa's Talking (sandbox)	Kenyan shortcodes; replay script as fallback
Hosting	API on Render or Railway, front-end on Vercel or Netlify	Deploys from GitHub in minutes

One Django API serves the app, the SMS webhook and the Signal panel



architecture · app, SMS gateway, API, database

App reports and inbound SMS both go through `process_report`; verified alerts go back out through the same gateway.

Data model

Eight tables. Every report is kept raw; an Event is what duplicates merge into, and only Events are shown to commuters.

Model	Key fields
User	phone (unique), created_at
Route	number, name ("Odeon → Githurai 45"), via, stages (M2M Stage, ordered)
Stage	name (Kencom, Ngara, Kasarani …), aliases (for SMS matching)
RouteSacco	route, sacco_name, usual_fare

Model	Key fields
Report	user or sender_phone, source (app / sms), raw_text, type, route, sacco, stage, fare, is_spam, spam_reason, event (FK, null), created_at
Event	type (fare_hike / jam / accident_police / route_closed), route, sacco, stage, fare, status (pending / verified / cleared / expired), reporter_count, first_seen, verified_at, expires_at
Vote	event, user, kind (still / cleared), unique per event + user
Alert	event, sent_at, channel (sms / push), routes_notified

Subscriptions (the bell toggle) can be an M2M from User to Route rather than its own table.

Core logic

One function, `process_report(report)` in `reports/services.py`, runs on every app report and inbound SMS. It is the demo, so it gets the unit tests.

1. **Parse** (SMS only; the app sends structured fields)
   - Type from keywords: fare / bei / KES → fare hike; jam / imejam / kwama → jam; acc / ajali / karao / polisi → accident or police; imefungwa / closed → route closed.
   - Stage by matching `Stage.aliases` ("g45" → Githurai 45). Fare = first 2–3 digit number. Sacco by name match.
2. **Spam check** → mark `is_spam` with a reason and stop if any rule hits:
   - Blocklist words: win, zawadi, loan, tuma pesa, click, any URL.
   - Nothing usable parsed (no type and no stage).
   - Over 5 reports from one phone in 10 min.
3. **Merge** into an open Event when route + type match, created in the last 30 min, and stage matches (fare hikes: same sacco, fare within KES 10). Otherwise create a new pending Event.
4. **Verify**: when distinct reporters on the Event reach 3 → status `verified`, set `verified_at`, write an Alert, notify route subscribers.

5. **Votes**: "Still accurate?" counts as a reporter and extends `expires_at`. 2 "Cleared?" votes → `cleared`.
6. **Expiry** on read: jam and police 60 min, accident 2 h, fare hike 6 h, route closed until cleared.

Derived on read:

- **Sacco fare**: latest verified fare-hike Event for that sacco, else `usual_fare`. A pending one shows as "KES 90?".
- **Route pill**: Kwama if a verified accident, closure or jam is active; Moderate if anything pending; else Maji Safi.

API

Ten endpoints under `/api/`. Agree the JSON shapes first so the front-end can build on mocks in parallel.

Method	Path	Does
POST	/auth/otp/request	Send a 4-digit code to `phone` (demo mode: always 4729)
POST	/auth/otp/verify	Check code, return JWT access + refresh
GET	/routes?search=	Routes with name, via and status pill
GET	/routes/{id}	Route, sacco fares, active Events, verified alert banner
POST	/routes/{id}/subscribe	Toggle the alerts bell
POST	/reports	Type, sacco?, fare?, note? → returns the Event and "N others reported this"
POST	/events/{id}/vote	`kind`: still or cleared
POST	/sms/inbound	Gateway webhook: from, text → `process_report`
GET	/signal	Funnel counts, latest raw reports, verified events, alert log
POST	/demo/replay	Staff only: replays the 200 seed SMS for the pitch

Front-end

One responsive React app serves both the mobile and web designs: same components, layout switches at the `md` breakpoint.

Page (path)	Mobile design	Web design
/login, /login/otp	1a, 1b	W1a, W1b
/routes	2 Route picker	W2–3 sidebar
/routes/:id	3 Route detail	W2–3 main panel
Report (state on /routes/:id)	4a/4b bottom sheet	W4a/W4b dialog
/signal	—	5 Signal panel

- **Tokens**: put the palette in `tailwind.config` (primary, ink, muted, alert, fare-up, fare-normal, pending + tints). Red stays reserved for verified alerts.
- **Components**: the 12 from the design system page, each one file in `src/components/`. Add `ReportForm` shared by the sheet and the dialog.
- **State**: TanStack Query for server data, JWT in localStorage, `refetchInterval: 15000` on route detail and Signal.
- **Optimistic UI**: votes and reports update the card immediately, then refetch.
- **Folders**: `pages/`, `components/`, `api/` (one file of fetch functions), `mocks/` (JSON matching the API table).

Team and build order

Assumes three people; with two, Backend A and B merge. Never let the front-end wait on the API: mocks first, swap to live endpoints per checkpoint.

Who	Owns
Backend A	Project setup, models, OTP auth, routes and subscribe endpoints, seed data, deploy
Backend B	`process_report` (parse, spam, merge, verify, votes, expiry) + tests, SMS webhook, Signal endpoint, replay script

Who	Owns
Front-end	Tailwind tokens, 12 components, all pages, API client, polling

Checkpoints, in order — each ends with a working demo:

1. **Skeleton**: repos, Django + Postgres up, React app on mocks, API JSON shapes agreed.
2. **Read path**: seeded routes, saccos and events served; route picker and route detail live.
3. **Write path**: report and votes live, verify-at-3 working end to end.
4. **Signal**: replay script fills the funnel to 200 → 61 → 27 → 12; Signal panel live.
5. **Ship**: deploy both apps, rehearse the demo twice, freeze code.

Demo and risks

The pitch runs on seeded data so nothing depends on live SMS.

Seed (`manage.py seed_demo`): routes 23, 46, 58, 111, 100 with their stages; four saccos on Route 100 (Thika Road Flyers, Mwangaza, Haraka Shuttle, Zuri Line); 200 SMS of which 61 are spam, merging into 27 events, 12 verified.

Demo script (3 min)

1. Sign in on a phone; open Route 100: Kwama pill, Ngara accident banner, Thika Road Flyers at KES 100.
2. Report a fare hike on a second phone; it shows "Thanks — 2 others reported this" and the card turns verified.
3. Switch to the laptop Signal panel: the funnel and struck-through spam tell the story.

Risk	Fallback
SMS gateway or OTP delivery fails on the day	Demo mode: fixed OTP 4729, replay script instead of live SMS
Sheng parsing misses messages	Keyword lists in one file, easy to extend; app reports skip parsing
One person faking 3 confirmations	Count distinct phones; rate limit per phone

Risk	Fallback
Venue Wi-Fi drops	Run both apps locally on one laptop with a phone on its hotspot